

Task 1 — Datasets + Project Plan

From Inside Game to Outside Game: Tracing the NBA's Three-Point Revolution, 2000–2026

Domain	Professional Basketball (NBA)	Course	Relational Databases — Group Project
Data source	nba_api (stats.nba.com)	Stack	PostgreSQL + Metabase
Scope	26 seasons · 7 tables · ~84,000 rows		

Team: Aditya Verma · Angela Wei · Disha Khati · Gursimrat Grewal · Yash Tiwari

One question, four layers of resolution: how did NBA shot selection move from the paint to the three-point line, which franchises moved first, and did moving early actually win games?

1 Finalized Domain

NBA basketball — how the game itself changed across the last 26 seasons. The NBA records every shot of every game and publishes it, which gives us a rare ability: to watch a sport transform itself, season by season, in the data.

2 Finalized Data Source

The NBA's own statistics API — stats.nba.com, accessed via the **nba_api** Python client (github.com/swar/nba_api, pip install nba_api). These are the undocumented JSON endpoints that power NBA.com's own stat pages — raw, untyped, and undocumented by the NBA itself. Free, no API key, but requires custom headers and self-imposed rate limiting (~1 request/sec).

Why a raw source, not a prepared dataset

We rejected packaged alternatives (Kaggle SQLite builds, etc.) because downloading a finished, normalized database would skip the part of the assignment that most resembles real data work: acquiring data never designed for our purposes and making it queryable ourselves. We design the schema, choose the keys, define the types, and handle the failures.

Scope: 26 seasons, 2000–01 through 2025–26

A bounded window, not all available history. Starting in 2000 gives roughly a decade of flat baseline before the three-point shift begins, making the inflection point far more visible than inside eighty years of sparse, inconsistent history.

The seven endpoints / tables

Endpoint	Becomes table	Rows	Purpose
LeagueGameLog (per season)	team_game	~68,400	One row per team per game, with box score
CommonTeamRoster (per team-season)	roster	~11,700	Who played where, per season
CommonPlayerInfo	player	~2,300	Biography, height, weight, position
LeagueDashTeamStats (per season)	team_season	~780	Aggregated team splits
TeamYearByYearStats	team_season_history	~780	Franchise year-by-year record
franchisehistory	team	~36	Franchise identity, relocations, renames

Endpoint	Becomes table	Rows	Purpose
(derived)	season	26	Era labels, rule changes, league context

Seven tables, ~84,000 rows total. Game-level context (date, home/away, season type) is folded into `team_game` rather than split out; season is the small dimension table our era-bucketing joins hang off.

The data is genuinely messy — this is the point

- **No types** — every value arrives as a JSON string or null; every column needs explicit casting.
- **Inconsistent column names** across endpoints (TEAM_ID vs TeamID vs TEAM_ABBREVIATION).
- **Unstable franchise identity** — relocations and renames (SuperSonics→Thunder, Nets, Hornets/Bobcats) require a defensible modeling decision.
- **Sparse, shifting coverage** — advanced/tracking columns appear only in later seasons; nulls mean different things in different places.
- **Mixed formats** — minutes as "38:00" vs 38.0; seasons as "2015-16"; inconsistent dates.
- **Duplicate/ambiguous player names** — `player_id` is authoritative but not always present.
- **Unreliable delivery** — timeouts, partial results, rate limiting; ingestion needs retry logic.

Pipeline as a deliverable

Ingestion (cached, rate-limited, retried) → staging load (raw JSON as text) → cleaning/typing layer (documented transformations) → validation (row counts, referential integrity, spot checks) → data quality report (what was dropped, inferred, or remains uncertain).

Core relationships

```
team(team_id) --< team_game, team_season, team_season_history, roster (all by team_id, season)
player(player_id) --< roster (by player_id)
```

3 Why We Chose This Domain

- **Raw data** → **we build the database ourselves**. The relational design work — schema, keys, resolved inconsistencies — is entirely ours.
- **Right scale for the question**. ~84,000 rows across 7 joined tables: large enough that unindexed multi-table joins are measurably slow, small enough to iterate in seconds.
- **Self-explanatory domain**. No briefing needed on what a basketball game is — attention goes to the analysis.
- **Verifiable results**. Basketball analytics is a mature public discipline we can check our numbers against.
- **26 continuous seasons invite a real question** — long enough to show a sport becoming a different sport, short enough to be recorded consistently throughout.

4 The Story: One Question

How did the NBA change from an inside game to an outside game — and did the teams that changed first actually win more? For most of NBA history, scoring happened near the basket. The three-point line arrived in 1979 and stayed a novelty for two decades — then around 2012 shot selection reorganized itself within about eight seasons, and the mid-range jumper nearly disappeared. The 2000–2026 window captures a decade of flat baseline plus the entire acceleration and its aftermath.

Four layers of the same question

Layer	Focus	Question	Technique
Layer 1	The league curve	3PT attempt rate, points/possession, and shot mix by season — locate the inflection point.	Aggregation, date extraction, ordered output
Layer 2	Who moved first	Rank all 30 franchises by 3PT rate vs. league average, per season.	Window functions — RANK(), AVG() OVER (PARTITION BY)
Layer 3	Did it work?	Join shooting profile to win % — do early adopters win more, that season and the next two?	Self-joins, LAG(), correlation
Layer 4	Who changed	Join roster + player to team shooting profiles by position/body type over eras.	Multi-table joins, conditional aggregation, era bucketing

Each layer is a harder query than the last, and each adds evidence to the same conclusion — the narrative arc and the technical difficulty curve are the same curve. Layer 4 is the most demanding: it depends on having correctly resolved franchise identity and position-labeling during cleaning.

Data coverage: Layers 1–3 have full box-score coverage across 2000–2026. Layer 4's roster/position data is complete for the window, though position labels are applied inconsistently in the earliest seasons — noted in the data quality report.

5 Final Goal

An interactive dashboard that tells one story — the NBA's shift from an inside game to an outside game — traced across 26 seasons, backed by a fully indexed relational database and a documented set of increasingly complex SQL queries.

- Ingestion pipeline: raw JSON pulls with caching, rate limiting, retry handling
- Documented cleaning layer (raw JSON → normalized schema) + data quality report
- Loaded, indexed database with a self-designed ER diagram
- Query library of ~25 queries — aggregation → window functions → self-joins → CTEs
- Interactive dashboard walking through all four layers as a single narrative
- Performance report: query timings and execution plans before/after indexing
- Written analysis and final presentation

Why one question is the right scope: pointing every query at the same target means each result builds on the last instead of starting over — a narrow question is a deeper project, not a smaller one.

6 Seven-Week Plan

Wk	Focus	Output
1	Ingestion script, pull & cache raw JSON, design schema	Raw data on disk, draft ER diagram
2	Clean, type, load to staging → normalized tables; validate	Loaded database, data quality report
3	Layers 1–2: league curve and team ranking	Core curve validated against published figures
4	Layers 3–4: win correlation and roster composition	Window-function and multi-join queries complete
5	Indexing and optimization	Performance report with timings and query plans
6	Dashboard build	Working dashboard
7	Write-up, rehearsal, buffer	Final report and presentation

Risks and mitigations

- **Ingestion runs long (largest risk)** → cache raw JSON in Week 1 before cleaning starts; fall back to a reduced range (2010–2026) if needed — the inflection point still sits inside it.
 - **Endpoints change/break/rate-limit** → all responses cached to disk on first pull; a prepared dataset can backfill one failed table without changing the schema.
 - **Cleaning decisions turn out wrong** → every transformation is documented and reversible; staging retains original untyped values.
 - **Scope judged too narrow** → Layers 1–3 alone already require window functions, self-joins, lagged comparisons, and multi-season aggregation on a self-designed schema.
 - **Indexing gains look trivial** → benchmark un-indexed multi-join era queries and report EXPLAIN plan output showing scans converted to index lookups.
-

7 Quick Start

1. Pull raw data — season-by-season pulls via nba_api's LeagueGameLog, cached to disk so nothing is re-pulled. **2. Inspect what arrived** — each response is a resultSet envelope (headers + untyped rowSet); comparing 2000-01 headers to 2024-25 headers is the cleaning spec. **3. Load, clean, then query** — once staging tables are typed and normalized, the core curve of the entire project is one query:

```
SELECT season,
       ROUND(100.0 * SUM(fg3a) / SUM(fga), 1) AS pct_of_shots_from_three
FROM team_game
WHERE fg3a IS NOT NULL
GROUP BY season
ORDER BY season;
```